What Lab M Pyth GDB Class repor L New
jplag.github.io/JPlag/comparison/21L-6114_A1.cpp/Source.cpp
Photos - Google Ph... YouTube Facebook WhatsApp Classes Temp Mail - Dispos... LESCO - Customer... Bill Payment

JPlag Comparison

back

Submission 1
21L-6114_A1.cpp

Submission 2
Source.cpp

Match %
26.12

Submission 1 File 1	Submission 2 File 2	Tokens
21L-6114_A1.cpp\21L-6114_A1.cpp (71 - 88)	Source.cpp\L22-7904.cpp (57 - 71)	18
21L-6114_A1.cpp\21L-6114_A1.cpp (192 - 201)	Source.cpp\L22-7904.cpp (19 - 25)	14
21L-6114_A1.cpp\21L-6114_A1.cpp (66 - 69)	Source.cpp\L22-7904.cpp (52 - 55)	13
21L-6114_A1.cpp\21L-6114_A1.cpp (211 - 215)	Source.cpp\L22-7904.cpp (161 - 167)	13
21L-6114_A1.cpp\21L-6114_A1.cpp (115 - 122)	Source.cpp\L22-7904.cpp (170 - 178)	12

Files of 21L-6114_A1.cpp

21L-6114_A1.cpp\21L-6114_A1.cpp

```
1 /*
2
3 Laiba Tariq
4 21L-6114
5 BS-SE
6 2C
7 */
8
9 #include <iostream>
10 #include <string>
11 #include <string.h>
12 #include <cstring>
13 using namespace std;
14
15 void tokenize(char* str, char**& dictionary, int& dictSize) {
16     // initialize variables
17     int i = 0, j = 0, k = 0;
18     dictSize = 0;
19     char* word = new char[50]; // assume max word length is 50 characters
20
21     // loop through string character array
22     while (str[i] != '\0' && k < 3000) {
23         if (str[i] == ' ' || str[i] == '.') { // check for end of word
24             if (j > 0) { // if word has been formed
25                 word[j] = '\0'; // add null terminator
26                 bool duplicate = false;
27                 // check if word is already in dictionary
28                 for (int l = 0; l < dictSize; l++) {
29
30                     if (strcmp(word, dictionary[l]) == 0) {                    // CHANG
31                         duplicate = true;
32                         break;
33                     }
34                 }
35                 // add word to dictionary if not a duplicate
36                 if (!duplicate) {
37                     dictionary[dictSize] = new char[j + 1]; // allocate memory for wo
38
39                     //                    for(int i=0; i<dictSize; i++)
40                     //                    {
41                     //
42                     //                                        }
43
44                     strcpy_s(dictionary[dictSize], j + 1, word); // copy word to dict
45                     dictSize++; // increment dictionary size
46                 }
47             }
48             // reset variables for next word
49             j = 0;
50             delete[] word;
51             word = new char[50];
52         }
53         else {
54             word[j] = str[i]; // add character to word
55             j++;
56         }
57         i++;
```

Files of Source.cpp

Source.cpp\L22-7904.cpp

```
1 #include<iostream>
2 #include<cstring>
3 using namespace std;
4 bool areEqual(char* w1, char* w2)
5 {
6     int i = 0;
7     while (w1[i] && w2[i])
8     {
9             if (w1[i] != w2[i])
10            {
11                    return false;
12            }
13            i++;
14    }
15    return (w1[i] == '\0' && w2[i] == '\0');
16 }
17 void checkunique(char**& data_dictionary, int& row)
18 {
19    int count = 0;
20    for (int i = 0; i < row; i++)
21    {
22            for (int j = i + 1; j < row; j++)
23            {
24                    if (areEqual(data_dictionary[i], data_dictionary[j]))
25                    {
26                            count++;
27                            for (int k = 0;k < strlen(data_dictionary[i]); k++)
28                            {
29                                    data_dictionary[i][k] = ' ';
30                            }
31                    }
32            }
33    }
34    row = row - count;
35 }
36
37 int main()
38 {
39    //task 1
40    const int Size = 2000;
41    char arr[Size];
42    cin.getline(arr, Size);
43    int row = 2, col = 1;
44    int s = strlen(arr);
45    for (int i = 0;i < s;i++)
46    {
47            if (arr[i] == ' ' || arr[i] == '.')
48            {
49                    row++;
50            }
51    }
52    char** data_dictionary = new char* [row];
53    for (int j = 0;j < row;j++)
54    {
55            data_dictionary[j] = new char[20];   // set maximus length of each word
```

4:51 PM
4/14/2023

What × | Lab M × | Pyth × | GDB × | Class × | repor × | L × | New ×
jplag.github.io/JPlag/comparison/21L-6114_A1.cpp/Source.cpp
Photos - Google Ph... | YouTube | Facebook | WhatsApp | Classes | Temp Mail - Dispos... | LESCO - Customer... | Bill Payment

JPlag Comparison

back

Submission 1
21L-6114_A1.cpp

Submission 2
Source.cpp

Match %
26.12

Submission 1 File 1	Submission 2 File 2	Tokens
21L-6114_A1.cpp\21L-6114_A1.cpp (71 - 86)	Source.cpp\L22-7904.cpp (57 - 71)	18
21L-6114_A1.cpp\21L-6114_A1.cpp (192 - 201)	Source.cpp\L22-7904.cpp (19 - 25)	14
21L-6114_A1.cpp\21L-6114_A1.cpp (66 - 69)	Source.cpp\L22-7904.cpp (52 - 55)	13
21L-6114_A1.cpp\21L-6114_A1.cpp (211 - 215)	Source.cpp\L22-7904.cpp (161 - 167)	13
21L-6114_A1.cpp\21L-6114_A1.cpp (115 - 122)	Source.cpp\L22-7904.cpp (170 - 178)	12

```cpp
    int j = 0;
    for (int i = 0; i <= (strlen(str)); i++)
    {
        // if space or NULL found, assign NULL into newString[ctr]
        if (str[i] == ' ' || str[i] == '\0')
        {
            newString[ctr][j] = '\0';
            ctr++;  //for next word
            j = 0;   //for next word, init index to 0
        }
        else
        {
            newString[ctr][j] = str[i];
            j++;
        }
    }

    return newString;
}

void inputSynonym(string++& syn, int& s, char++& dictionary, int& dictSize)
{
    syn = new string * [dictSize];
    s = 0;  //s will be size of total number of synonyms stored

    int option, SynNum;

    for (int i = 0; i < dictSize; i++) {
        cout << "Enter 1 to store Synonyms for the word: " << dictionary[i] << endl;
        cin >> option;

         if (option == 1)
         {
             cout << "How many Synonyms you want to store" << endl;
             cin >> SynNum;
             syn[s] = new string[SynNum + 2]; // 2 extra indexes, 1 for word and other
             //copting word at 0 index
             syn[s][0] = dictionary[i];
             cout << "Word: " << syn[s][0] << endl;
             for (int j = 1; j < SynNum + 1; j++) {
                 cout << "Enter synonym " << j << " :";
                 cin >> syn[s][j];
             }
             syn[s][SynNum + 1] = "#";
             s++;

         }
    }

    cout << "Words and Synonyms" << endl;
    for (int i = 0; i < s; i++)
    {
        for (int j = 0; syn[i][j] != "#"; j++)
        {

            cout << syn[i][j] << " ";
        }
        cout << endl;
    }

}

//void search(string++ syn, int s)
//{
//    string arr2;
//    int j = 0;
//    string* arr3 = new string;
//    int count = 0;
//    cout << "Input word(function): ";
//    cin.ignore(); //to clear buffer
```

```cpp
    int r = 0, c = 0;
    for (int i = 0; arr[i] != '\0'; i++)
    {
            if (arr[i] == ' ' || arr[i] == '.')
            {
                    data_dictionary[r][c] = '\0'; // end of word
                    r++;
                    c = 0;
            }
            else
            {
                    data_dictionary[r][c] = arr[i];
                    c++;
            }
    }

    data_dictionary[r][c] = '\0';
    //check for unique words
    checkunique(data_dictionary, row);
    cout << endl;
    cout << "____________Tokens:_____________ " << endl;
    for (int i = 0; i < row-1; i++)
    {
            cout << data_dictionary[i] << endl;
    }
    cout << endl;
    //task 2
    char+++ synonyms = new char++ [row];
    for (int i = 0;i < row;i++)
    {
            synonyms[i] = new char+ [row];
            for (int j = 0;j < row;j++)
            {
                    synonyms[i][j] = new char[row];

                    for (int k = 0;k < row;k++)
                    {
                            synonyms[i][j][k] = NULL;
                    }
            }
    }
    const int x = 100;
    int counts[x];
    for (int i = 0; i < row; i++)
    {
            counts[i] = 0;
            cout << "the " << i + 1 << " word of data dictionary is : " << data_dicti
            char response;
            cout << "Do you want to store synonyms for this word? (y/n): ";
            cin >> response;

            if (response == 'y')
            {
                    cout << "Enter the number of synonyms: ";
                    cin >> counts[i];
                    for (int j = 0; j < counts[i]; j++)
                    {
                            char temp[x];
                            cout << "Enter synonym " << j + 1 << ": ";
                            cin >> temp;

                            // Allocate memory for synonym in 3D array
                            synonyms[i][j] = new char[strlen(temp) + 1];
                            synonyms[i][j] = temp;
                    }
            }
    }
    //task 3
    int Siz = 2000;
    char+ usefase1 = new char[Siz];
```

4:51 PM
4/14/2023

What × | Lab M × | Pyth × | GDB × | Class × | repor × | L × | New ×
jplag.github.io/JPlag/comparison/21L-6114_A1.cpp/Source.cpp
Photos - Google Ph... | YouTube | Facebook | WhatsApp | Classes | Temp Mail - Dispos... | LESCO - Customer... | Bill Payment

JPlag Comparison

back

Submission 1
21L-6114_A1.cpp

Submission 2
Source.cpp

Match %
26.12

Submission 1 File 1	Submission 2 File 2	Tokens
21L-6114_A1.cpp\21L-6114_A1.cpp (71 - 86)	Source.cpp\L22-7904.cpp (57 - 71)	18
21L-6114_A1.cpp\21L-6114_A1.cpp (192 - 201)	Source.cpp\L22-7904.cpp (19 - 25)	14
21L-6114_A1.cpp\21L-6114_A1.cpp (66 - 69)	Source.cpp\L22-7904.cpp (52 - 55)	13
21L-6114_A1.cpp\21L-6114_A1.cpp (211 - 215)	Source.cpp\L22-7904.cpp (161 - 167)	13
21L-6114_A1.cpp\21L-6114_A1.cpp (115 - 122)	Source.cpp\L22-7904.cpp (170 - 178)	12

```
192     int arrSize = 0;
193     //creating 2D array of uniqueWords form useCase Array
194     tokenize(input, uniqueWords, arrSize);
195
196     cout << endl << endl;
197     cout << "Displaying useCase1 Words in Dictionary" << endl;
198     for (int i = 0; i < arrSize; i++) {
199         for (int j = 0; j < dictSize; j++) {
200             //comparing and displaying if similar
201             if (!strcmp(uniqueWords[i], dictionary[j])) {
202                 cout << uniqueWords[i] << endl;
203                 break;
204             }
205         }
206     }
207
208     //task4
209     int sizeTokenizedInput;
210     //Tokenizing without removing repeated words.
211     char** tokenizedInput = tokenize(input, sizeTokenizedInput);
212     cout << endl << endl;
213     for (int i = 0; i < sizeTokenizedInput; i++) {
214         for (int j = 0; j < s; j++) {
215             if (tokenizedInput[i] == syn[j][0]) {
216                 int k = 1;
217                 cout << syn[j][0] << " has Synonyms:  ";
218                 while (syn[j][k] != "#")
219                     cout << syn[j][k++] << " ";
220                 cout << endl;
221                 //changing the word with Synonym by Coping the 1st synonym
222                 Copy(tokenizedInput[i], syn[j][1]);
223                 break;
224             }
225         }
226     }
227     //Displaying final output
228     cout << "Replacing with Synonyms" << endl;
229     for (int i = 0; i < sizeTokenizedInput; i++) {
230         cout << tokenizedInput[i] << " ";
231     }
232     cout << endl;
233
234     //deallocation
235     for (int i = 0; i < sizeTokenizedInput; i++)
236         delete[]tokenizedInput[i];
237     delete[]tokenizedInput;
238     for (int i = 0; i < arrSize; i++)
239         delete[]uniqueWords[i];
240     delete[]uniqueWords;
241     delete[]input;
242 }
243
244 int main() {
245     // get input from user
246     char* input = new char[3001]; // assume max input length is 3000 characters
247     cout << "Enter Sentence to be tokenized: ";
248     cin.getline(input, 3001);
249
250     // tokenize input and store in data dictionary
251     char** dictionary = new char* [1000]; // assume max unique words is 1000
252     int dictSize = 0;
253     tokenize(input, dictionary, dictSize);
254
255     // print data dictionary
256     cout << "\nOutput Data Dictionary:\n";
257     for (int i = 0; i < dictSize; i++)
258     {
```

```
19      int count = 0;
20      for (int i = 0; i < row; i++)
21      {
22              for (int j = i + 1; j < row; j++)
23              {
24                      if (areEqual(data_dictionary[i], data_dictionary[j]))
25                      {
26                              count++;
27                              for (int k = 0;k < strlen(data_dictionary[i]); k++)
28                              {
29                                      data_dictionary[i][k] = ' ';
30                              }
31                      }
32              }
33      }
34      row = row - count;
35 }
36
37 int main()
38 {
39      //task 1
40      const int Size = 2000;
41      char arr[Size];
42      cin.getline(arr, Size);
43      int row = 2, col = 1;
44      int s = strlen(arr);
45      for (int i = 0;i < s;i++)
46      {
47              if (arr[i] == ' ' || arr[i] == '.')
48              {
49                      row++;
50              }
51      }
52      char** data_dictionary = new char* [row];
53      for (int j = 0;j < row;j++)
54      {
55              data_dictionary[j] = new char[20];   // set maximum length of each word t
56      }
57      int r = 0, c = 0;
58      for (int i = 0; arr[i] != '\0'; i++)
59      {
60              if (arr[i] == ' ' || arr[i] == '.')
61              {
62                      data_dictionary[r][c] = '\0'; // end of word
63                      r++;
64                      c = 0;
65              }
66              else
67              {
68                      data_dictionary[r][c] = arr[i];
69                      c++;
70              }
71      }
72      data_dictionary[r][c] = '\0';
73      //check for unique words
74      checkunique(data_dictionary, row);
75      cout << endl;
76      cout << "____________Tokens:_______________ " << endl;
77      for (int i = 0; i < row-1; i++)
78      {
79              cout << data_dictionary[i] << endl;
80      }
81      cout << endl;
82      //task 2
83      char*** synonyms = new char** [row];
```

4:51 PM
4/14/2023

What × | Lab M × | Pytho × | GDB × | Class × | repor × | L × | New ×
jplag.github.io/JPlag/comparison/21L-6114_A1.cpp/Source.cpp
Photos - Google Ph... | YouTube | Facebook | WhatsApp | Classes | Temp Mail - Dispos... | LESCO - Customer... | Bill Payment

JPlag Comparison

back

Submission 1
21L-6114_A1.cpp

Submission 2
Source.cpp

Match %
26.12

Submission 1 File 1	Submission 2 File 2	Tokens
21L-6114_A1.cpp\21L-6114_A1.cpp (71 - 86)	Source.cpp\L22-7904.cpp (57 - 71)	18
21L-6114_A1.cpp\21L-6114_A1.cpp (192 - 201)	Source.cpp\L22-7904.cpp (19 - 25)	14
21L-6114_A1.cpp\21L-6114_A1.cpp (66 - 69)	Source.cpp\L22-7904.cpp (52 - 55)	13
21L-6114_A1.cpp\21L-6114_A1.cpp (211 - 215)	Source.cpp\L22-7904.cpp (161 - 167)	13
21L-6114_A1.cpp\21L-6114_A1.cpp (115 - 122)	Source.cpp\L22-7904.cpp (170 - 178)	12

```cpp
    char** newString = new char* [1000]; //max words 1000
    for (int i = 0; i < 10; i++) {
        newString[i] = new char[50];
    }

    ctr = 0;

    int j = 0;
    for (int i = 0; i <= (strlen(str)); i++)
    {
        // if space or NULL found, assign NULL into newString[ctr]
        if (str[i] == ' ' || str[i] == '\0')
        {
            newString[ctr][j] = '\0';
            ctr++;  //for next word
            j = 0;   //for next word, init index to 0
        }
        else
        {
            newString[ctr][j] = str[i];
            j++;
        }
    }

    return newString;
}

void inputSynonym(string**& syn, int& s, char**& dictionary, int& dictSize)
{
    syn = new string * [dictSize];
    s = 0;  //s will be size of total number of synonyms stored

    int option, SynNum;

    for (int i = 0; i < dictSize; i++) {
        cout << "Enter 1 to store Synonyms for the word: " << dictionary[i] << endl;
        cin >> option;

         if (option == 1)
         {
             cout << "How many Synonyms you want to store" << endl;
             cin >> SynNum;
             syn[s] = new string[SynNum + 2]; // 2 extra indexes, 1 for word and other
             //copting word at 0 index
             syn[s][0] = dictionary[i];
             cout << "Word: " << syn[s][0] << endl;
             for (int j = 1; j < SynNum + 1; j++) {
                 cout << "Enter synonym " << j << " :";
                 cin >> syn[s][j];
             }
             syn[s][SynNum + 1] = "#";
             s++;

         }
     }

     cout << "Words and Synonyms" << endl;
     for (int i = 0; i < s; i++)
     {
         for (int j = 0; syn[i][j] != "#"; j++)
         {

             cout << syn[i][j] << " ";
         }
         cout << endl;
     }

}

//void search(string** syn, int s)
///
```

```cpp
    char** data_dictionary = new char* [row];
    for (int j = 0;j < row;j++)
    {
            data_dictionary[j] = new char[20];   // set maximum length of each word t
    }

    int r = 0, c = 0;
    for (int i = 0; arr[i] != '\0'; i++)
    {
            if (arr[i] == ' ' || arr[i] == '.')
            {
                    data_dictionary[r][c] = '\0'; // end of word
                    r++;
                    c = 0;
            }
            else
            {
                    data_dictionary[r][c] = arr[i];
                    c++;
            }
    }

    data_dictionary[r][c] = '\0';
    //check for unique words
    checkunique(data_dictionary, row);
    cout << endl;
    cout << "_____Tokens:_____ " << endl;
    for (int i = 0; i < row-1; i++)
    {
            cout << data_dictionary[i] << endl;
    }
    cout << endl;
    //task 2
    char*** synonyms = new char** [row];
    for (int i = 0;i < row;i++)
    {
            synonyms[i] = new char* [row];
            for (int j = 0;j < row;j++)
            {
                    synonyms[i][j] = new char[row];

                    for (int k = 0;k < row;k++)
                    {
                            synonyms[i][j][k] = NULL;
                    }
            }
    }
    const int x = 100;
    int counts[x];
    for (int i = 0; i < row; i++)
    {
            counts[i] = 0;
            cout << "the " << i + 1 << " word of data dictionary is : " << data_dicti
            char response;
            cout << "Do you want to store synonyms for this word? (y/n): ";
            cin >> response;

            if (response == 'y')
            {
                    cout << "Enter the number of synonyms: ";
                    cin >> counts[i];
                    for (int j = 0; j < counts[i]; j++)
                    {
                            char temp[x];
                            cout << "Enter synonym " << j + 1 << ": ";
                            cin >> temp;

                            // Allocate memory for synonym in 3D array
                            synonyms[i][j] = new char[strlen(temp) + 1];
```

4:51 PM
4/14/2023

What Lab M Pytho GDB Class repor L New
jplag.github.io/JPlag/comparison/21L-6114_A1.cpp/Source.cpp
Photos - Google Ph... YouTube Facebook WhatsApp Classes Temp Mail - Dispos... LESCO - Customer... Bill Payment

JPlag Comparison

back

Submission 1
21L-6114_A1.cpp

Submission 2
Source.cpp

Match %
26.12

Submission 1 File 1	Submission 2 File 2	Tokens
21L-6114_A1.cpp\21L-6114_A1.cpp (71 - 85)	Source.cpp\L22-7904.cpp (57 - 71)	18
21L-6114_A1.cpp\21L-6114_A1.cpp (192 - 201)	Source.cpp\L22-7904.cpp (19 - 25)	14
21L-6114_A1.cpp\21L-6114_A1.cpp (66 - 69)	Source.cpp\L22-7904.cpp (52 - 55)	13
21L-6114_A1.cpp\21L-6114_A1.cpp (211 - 215)	Source.cpp\L22-7904.cpp (161 - 167)	13
21L-6114_A1.cpp\21L-6114_A1.cpp (115 - 122)	Source.cpp\L22-7904.cpp (170 - 178)	12

```
211     char** tokenizedInput = tokenize(input, sizeTokenizedInput);
212     cout << endl << endl;
213     for (int i = 0; i < sizeTokenizedInput; i++) {
214         for (int j = 0; j < s; j++) {
215             if (tokenizedInput[i] == syn[j][0]) {
216                 int k = 1;
217                 cout << syn[j][0] << " has Synonyms:  ";
218                 while (syn[j][k] != "#")
219                     cout << syn[j][k++] << " ";
220                 cout << endl;
221                 //changing the word with Synonym by Coping the 1st synonym
222                 Copy(tokenizedInput[i], syn[j][1]);
223                 break;
224             }
225         }
226     }
227     //Displaying final output
228     cout << "Replacing with Synonyms" << endl;
229     for (int i = 0; i < sizeTokenizedInput; i++) {
230         cout << tokenizedInput[i] << " ";
231     }
232     cout << endl;
233
234     //deallocation
235     for (int i = 0; i < sizeTokenizedInput; i++)
236         delete[]tokenizedInput[i];
237     delete[]tokenizedInput;
238     for (int i = 0; i < arrSize; i++)
239         delete[]uniqueWords[i];
240     delete[]uniqueWords;
241     delete[]input;
242 }
243
244 int main() {
245     // get input from user
246     char* input = new char[3001]; // assume max input length is 3000 characters
247     cout << "Enter Sentence to be tokenized: ";
248     cin.getline(input, 3001);
249
250     // tokenize input and store in data dictionary
251     char** dictionary = new char* [1000]; // assume max unique words is 1000
252     int dictSize = 0;
253     tokenize(input, dictionary, dictSize);
254
255     // print data dictionary
256     cout << "\nOutput Data Dictionary:\n";
257     for (int i = 0; i < dictSize; i++)
258     {
259         cout << dictionary[i] << endl;
260     }
261
262     //s should be equal to dictSize
263     int s = 0;
264     string** syn;
265     inputSynonym(syn, s, dictionary, dictSize);
266     //myToken(data, list, rows);
267     search(syn, s, dictionary, dictSize);
268
269
270     // memory deallocation of input and data dictionary
271     delete[] input;
272     for (int i = 0; i < dictSize; i++) {
273         delete[] dictionary[i];
274         //delete[] syn[i];
275     }
276     for (int i = 0; i < s; i++)
277         delete[]syn[i];
278     delete[] dictionary;
279     delete[] syn;
```

```
141         temp[j] = new char[20];
142     }
143
144
145     int n = 0, m = 0;
146     for (int i = 0; useCase1[i] != '\0'; i++)
147     {
148         if (useCase1[i] == ' ' || useCase1[i] == '.')
149         {
150             temp[n][m] = '\0';
151             n++;
152             m = 0;
153         }
154         else
155         {
156             temp[n][m] = useCase1[i];
157             m++;
158         }
159     }
160     temp[n][m] = '\0';
161     int count = 0;
162     for (int i = 0; i < n; i++)
163     {
164         for (int j = 0; j < row; j++)
165         {
166             if (areEqual(temp[i], data_dictionary[j]))
167             {
168                 count++;
169             }
170         }
171     }
172     cout << "these are " << count << " words in data dictionary" << endl;
173     for (int i = 0; i < n; i++)
174     {
175         for (int j = 0; j < row; j++)
176         {
177             if (areEqual(temp[i], data_dictionary[j]))
178             {
179                 cout << temp[i] << endl;
180             }
181         }
182     }
183     //task 4
184     char resp;
185     for (int i = 0; i < n; i++)
186     {
187         for (int j = 0; j < row; j++)
188         {
189             if (areEqual(temp[i], data_dictionary[j]))
190             {
191                 resp = 'y';
192             }
193             else
194             {
195                 resp = 'n';
196             }
197         }
198     }
199
200     system("pause");
201     return 0;
202 }
```

4:51 PM
4/14/2023

What × | Lab M × | Pytho × | GDB × | Class × | repor × | L × | New × | +

jplag.github.io/JPlag/comparison/21L-6114_A1.cpp/Source.cpp

Photos - Google Ph... | YouTube | Facebook | WhatsApp | Classes | Temp Mail - Dispos... | LESCO - Customer... | Bill Payment

JPlag Comparison

back

Submission 1
21L-6114_A1.cpp

Submission 2
Source.cpp

Match %
26.12

Submission 1 File 1	Submission 2 File 2	Tokens
21L-6114_A1.cpp\21L-6114_A1.cpp (71 - 85)	Source.cpp\L22-7904.cpp (57 - 71)	18
21L-6114_A1.cpp\21L-6114_A1.cpp (192 - 201)	Source.cpp\L22-7904.cpp (19 - 25)	14
21L-6114_A1.cpp\21L-6114_A1.cpp (66 - 69)	Source.cpp\L22-7904.cpp (52 - 55)	13
21L-6114_A1.cpp\21L-6114_A1.cpp (211 - 215)	Source.cpp\L22-7904.cpp (161 - 167)	13
21L-6114_A1.cpp\21L-6114_A1.cpp (115 - 122)	Source.cpp\L22-7904.cpp (170 - 178)	12

```
115         }
116     }
117
118     cout << "Words and Synonyms" << endl;
119     for (int i = 0; i < s; i++)
120     {
121         for (int j = 0; syn[i][j] != '#'; j++)
122         {
123             cout << syn[i][j] << " ";
124         }
125         cout << endl;
126     }
127
128 }
129
130 //void search(string* syn, int s)
131 //{
132 //    string arr2;
133 //    int j = 0;
134 //    string* arr3 = new string;
135 //    int count = 0;
136 //    cout << "Input word(function): ";
137 //    cin.ignore(); //to clear buffer
138 //    getline(cin, arr2);
139 //    for (int i = 0; arr2[i] != '\0'; i++)
140 //    {
141 //        if (arr2[i] == ' ')
142 //        {
143 //            continue;
144 //        }
145 //        else
146 //        {
147 //            (*arr3)[count] = arr2[i];
148 //            cout << endl;
149 //            count++;
150 //        }
151 //    }
152 //    (*arr3)[count] = '\0';
153 //    cout << *arr3 << endl;
154 //    _getch();
155 //    count = 0;
156 //    for (int i = 0; i < count; i++)
157 //    {
158 //        for (int j = 0; j < s; j++)
159 //        {
160 //            if (arr3 == syn[j])
161 //            {
162 //                count++;
163 //
164 //                cout << "The synonym of ";
165 //                cout << syn[j][0] << " is: " << syn[j][1];
166 //            }
167 //        }
168 //    }
169 //    if (count == 0)
170 //    {
171 //        cout << "No synonyms found";
172 //    }
173 //}
174
175 void Copy(char*& arr, string& s) {
176     int i;
177     for (i = 0; i < s.size(); i++) {
178         arr[i] = s[i];
179     }
180     arr[i] = '\0';
181 }
182
183 void search(string* syn, int s, char**& dictionary, const int dictSize)
```

```
141         temp[j] = new char[20];
142     }
143
144
145     int n = 0, m = 0;
146     for (int i = 0; useCase1[i] != '\0'; i++)
147     {
148             if (useCase1[i] == ' ' || useCase1[i] == '.')
149             {
150                     temp[n][m] = '\0';
151                     n++;
152                     m = 0;
153             }
154             else
155             {
156                     temp[n][m] = useCase1[i];
157                     m++;
158             }
159     }
160     temp[n][m] = '\0';
161     int count = 0;
162     for (int i = 0; i < n; i++)
163     {
164             for (int j = 0; j < row; j++)
165             {
166                     if (areEqual(temp[i], data_dictionary[j]))
167                     {
168                             count++;
169                     }
170             }
171     }
172     cout << "these are " << count << " words in data dictionary" << endl;
173     for (int i = 0; i < n; i++)
174     {
175             for (int j = 0; j < row; j++)
176             {
177                     if (areEqual(temp[i], data_dictionary[j]))
178                     {
179                             cout << temp[i] << endl;
180                     }
181             }
182     }
183     //task 4
184     char resp;
185     for (int i = 0; i < n; i++)
186     {
187             for (int j = 0; j < row; j++)
188             {
189                     if (areEqual(temp[i], data_dictionary[j]))
190                     {
191                             resp = 'y';
192                     }
193                     else
194                     {
195                             resp = 'n';
196                     }
197             }
198     }
199
200     system("pause");
201     return 0;
202 }
```

4:51 PM
4/14/2023